

Bayes Networks

CSC 480: Artificial Intelligence, Spring 2026
Professor Austin P. Wright: Course Notes

Abstract

We will cover the visual language of BayesNets (or Probabilistic Graphical Models), and how they are used to model complex probabilistic relationships.

I Bayes Networks

If we have a joint distribution for some set of random variables, we can answer any question about probabilities by inference through enumeration. However if there are n random variables, each with a sample space of size d , then the joint distribution table must have d^n entries, which in general we cannot hope to keep in memory for any nontrivial problem.

However, if we know something about the domain, and thus the conditional independence of the different variables, we can save an enormous amount of space by *factoring* the joint distribution in terms of conditional distributions. A Bayes Net (or Probabilistic Graphical Model) gives us a *language for specifying a conditional factorization structure of the joint distribution*.

Definition: Bayes Net

A Bayes Net consists of:

1. A *directed acyclic graph* of nodes, where each node corresponds to a single random variable (X).
2. A *conditional probability distribution* for each node $P(X \mid \text{parents}(X))$

Which together encode all of the information of the entire joint distribution (i.e. how the joint distribution factorizes in terms of conditional distributions)

Thus for the set of N random variables, the joint distribution can be written as

$$P(X_1, X_2, \dots, X_N) = \prod_{i=1}^N P(X_i \mid \text{parents}(X_i)) \tag{1}$$

The structure of a Bayes net graph encodes the conditional independence of the constituent variables. It is these conditional independence assumptions that allows the storage of multiple small probability tables instead of one huge table.

While encoding conditional independence and conditional distributions, remember that there is no sense in which edges in a Bayes Net must refer to a *causal* relationship, just that there is *some* relationship. Furthermore, these decisions about relationships (or lack thereof) are all *modeling choices* that enable real world problem solving. But since **all models are wrong**, since they are necessarily simplifications, we must very carefully interrogate these simplifications.

Contents

I	Bayes Networks	1
	Example	2
	Conditional Independence in Bayes Nets	3
II	D-separation	4
	Common Structures (Triples) . . .	4
	D-separation Algorithm	4
III	Exact Inference: Variable Elimination	6
	Ordering	6
	Example	7

Discussion

Consider two models of the same two variables, rain and traffic. In one model we draw both variables as disconnected and independent. In the other we model rain as linked toward traffic. If an agent were to use either of those models as part of its probabilistic reasoning, which might you expect to be more useful? Also consider the other direction, with traffic linked toward rain, what is the difference? Think about *causality* and how it is useful but not enforced (especially if there may be missing variables).

Example

Consider an example of a Bayes net of slightly expanded problem from the previous lecture. Here the random variables are:

- B : Representing if a burglary occurs
- E : Representing if an earthquake occurs
- F : Representing if a fire occurs
- S : Representing if a smoke occurs
- A : Representing if a the alarm goes off
- J : Representing if Jeff calls 911
- M : Representing if Musty calls 911

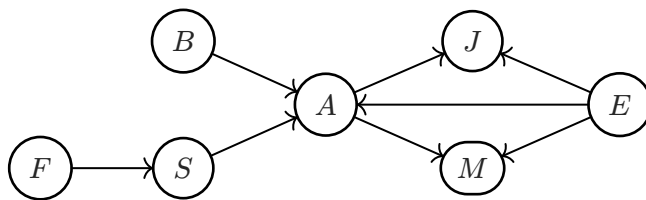


Figure 1: Example of Bayes Net

Discussion

Write out the joint distribution in terms of the conditional distributions according to the provided Bayes Net.

Conditional Independence in Bayes Nets

If the conditionals of a Bayes Net do in fact factorize the joint distribution, then the graph structure of the network allows us to make a number of claims of the conditional independence of the variables (this, essentially, is just the interpretation of *what we actually mean when we make a model using a Bayes net*). We have that:

1. Each node is conditionally independent of all ancestor nodes¹ in the graph, given all of its parents.
2. Each node is conditionally independent of *all other variables* given its *Markov Blanket*

¹Importantly, ancestors include *all non descendants* which could, in principle, be earlier than X in the DAG ordering.

Definition: Markov Blanket

A node's Markov blanket is the set of all of a node's parents, children, and other parents of it's children.

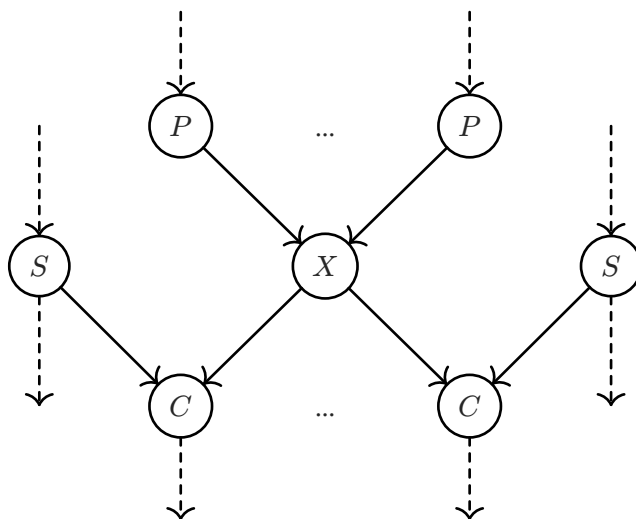


Figure 2: Markov Blanket for X

Discussion

Examine the earlier example of the alarm. Note how when writing the joint distribution in terms of the Bayes Net (i.e. $P(x \mid \text{parents})$), and then writing the joint in terms of the chain rule (i.e. $P(x \mid \text{parents, ancestors})$), the equation holds precisely as the conditional independence rules above hold.

II D-separation

D-separation (direct separation) is the process by which we can query the structure of a Bayes Net to answer questions about the conditional independence between arbitrary variables given arbitrary variables. Note that while the topology of the graph can communicate when *variables must be conditionally independent*, it does not enforce the other direction. It is possible that variables which the network considers possibly dependent may be, in reality, conditionally independent.

Common Structures (Triples)

While there is only two kinds of two variable relationships in a network (connected or disconnected), we can categorize a larger set of three node structures that will help us understand what we are looking at in a Bayes Net.



Figure 3: Causal Chain Structure

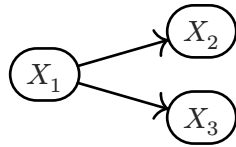


Figure 4: Common Cause Structure

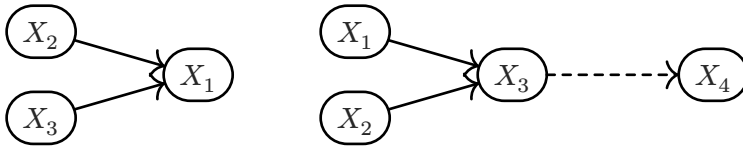


Figure 5: Common Effect Structure

D-separation Algorithm

1. Shade all observed nodes $\{Z_1, \dots, Z_k\}$ in the graph.
2. Enumerate all undirected paths from X to Y .
3. For each path:
 1. Decompose the path into triples.
 2. If all triples are open, the path d-connects X to Y .
4. If no path d-connects X and Y , then $X \perp\!\!\!\perp Y \mid \{Z_1, \dots, Z_k\}$ ².

²You can also derive these triples from the rules where a path is closed if either:

1. The arrows on the path meet either head-to-tail or tail-to-tail at a colored node
2. The arrows meet head-to-head at the node, and neither the node, nor any of its descendants is colored.

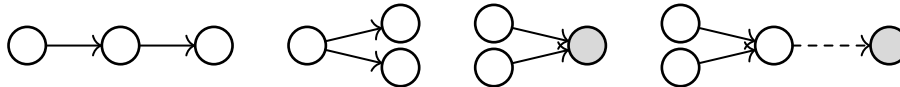


Figure 6: Open Triples

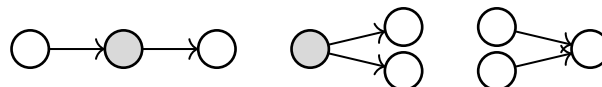
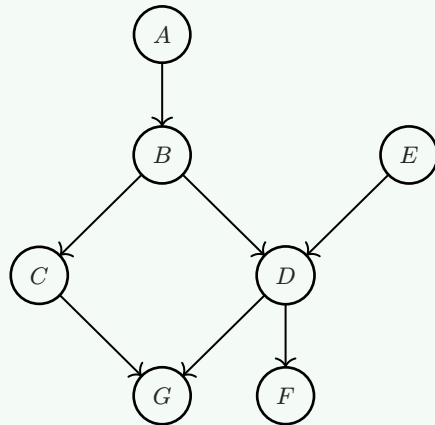


Figure 7: Closed Triples

Discussion

For the provided BayesNet, which of the following conditional independences *must* hold?

$$A \perp\!\!\!\perp F \mid D$$

$$A \perp\!\!\!\perp E$$

$$A \perp\!\!\!\perp E \mid D$$

$$A \perp\!\!\!\perp E \mid F$$

$$A \perp\!\!\!\perp E \mid D, B$$

$$A \perp\!\!\!\perp E \mid D, B$$

$$C \perp\!\!\!\perp D$$

$$C \perp\!\!\!\perp D \mid B$$

$$C \perp\!\!\!\perp D \mid B, G$$

(2)

III Exact Inference: Variable Elimination

The primary task our agents might have when using Bayes Nets to model the world is *inference*, that is finding some arbitrary new conditional probability from the joint (generally as a result of conditioning on new observation data). We already know how to do this by using the Bayes Net to create the joint distribution and then using inference by enumeration, but this is intractable. Luckily we can do better while maintaining an *exact* answer (i.e. always precisely the same probability as would be generated by full join inference), using a technique called *Variable Elimination*³.

Definition: Variable Elimination

To *eliminate* a variable X we simply:

1. Join all *factors* involving X
2. Sum out X

Where a *factor* is simply an *unnormalized probability* given by selecting entries from the conditional probability table consistent with the evidence.

³While this method is much more efficient than the exponential inference by enumeration, it is still NP-Hard and so while there are some problems for which it is tractable, often we will need to use approximate solutions which will be covered next

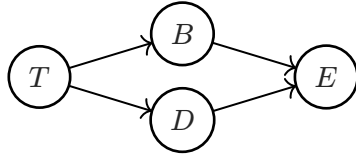
```
def eliminate(x: Query, e: Evidence, bn: BayesNet):
    factors = []
    for var in bn.Variables:
        factors = [MAKE_FACTOR(var,e)] + factors
        if is_hidden_variable(var):
            factors = SUM_OUT(var,factors)
    return NORMALIZE(PRODUCT(factors))
```

Ordering

Note that, like many of the algorithms in this class, there is a component where we need to iterate over the variables in the BayesNet in some non-deterministically defined order. It turns out that the order makes a massive impact on the efficiency of variable elimination, and heuristics are used here to choose what we think will be good orders. Such heuristics include *minimum degree* where we prioritize eliminating variables which results in constructing the smallest factor possible, and *minimum fill* which constructs an undirected graph of variable relations and eliminates the variable that would result in the fewest edges added after elimination.

Example

Suppose we have the model shown below. Where a wizard has an opportunity to take a golden statue from a suspicious pedestal in some ancient ruins. Let us say T is the random variable representing if the wizard takes the statue. Then B is a random variable representing if a boulder falls from the ceiling, D represents darts being shot from the walls, and E represents the wizard escaping the ruins. We then have the BayesNet below:



This gives us the conditional probability distributions:

$$P(T), P(B | T), P(D | T), P(E | B, D) \quad (3)$$

Suppose we want to calculate the probability that the wizard took the statue, given that it was able to escape: $P(T | e)$. Using variable elimination we don't have to calculate the whole 16 row joint, instead we can eliminate B then D which is done as follows:

1. Join (multiply) factors involving B , giving:

$$f_1(T, B, D, e) = P(B|T) \cdot P(e | B, D) \quad (4)$$

1. Sum out B from this new factor, leaving $f_2(T, D, e)$
2. Join factors involving D forming:

$$f_3(T, D, e) = P(D|T) \cdot f_2(T, D, e) \quad (5)$$

1. Sum out D , giving $f_4(T, e)$.
2. Join the remaining factors, which gives:

$$f_5(T, e) = f_4(T, e) \cdot P(T) \quad (6)$$

1. Once we have $f_5(T, e)$, we can find $P(T|e)$ by normalizing.

One thing that we can see is how variable elimination improves over inference by enumeration. In inference by enumeration we must first join *all* of the conditional distributions, then sum out all of the variables:

$$\sum_b \sum_d P(T) P(b|T) P(d|T) P(e|b, d) \quad (7)$$

But in variable elimination we are able to simply move the terms that are constant in a sum outside of the sum, reducing the size of the inner tables:

$$P(T) \sum_b P(b|T) \sum_d P(d|T) P(e|b, d) \quad (8)$$